Final project

Upload your final project on canvas before **December, 17** 2019, 11:59pm. **This is a HARD deadline**.
Upload **only one file**, e.g., a ZIP or a TAR file (or an equivalent archive file) that contains your final project.
Comment your code so that a non expert can understand your code.

The goal of the final project is to *implement* a convex optimization method that allows us to enhance the quality of a digital image by removing "noise"/perturbations. This imaging problem is called the Rudin-Osher-Fatemi problem. The model and the minimimzation algorithm has been described in class. Also, I remind you the 1 dimensional case has been covered in class (i.e., an image has only "one" line).

For this project, we consider that an image is a 2 dimensional grid with M rows and N columns, where each entry is a real value (therefore, we consider grayscale images). We denote by $y_{i,j}$ the value of the image at the site (i,j) where $1 \leq i \leq M$ and $1 \leq j \leq N$. For mathematical purposes we see an image y as a vector of $\mathbb{R}^n$ where $n = MN$. Therefore, we abuse notation and also denote by $y_{i,j}$ an entry of this vector.

Let $x \in \mathbb{R}^n$ be an observed image. Let $t > 0$. We wish to minimize for $y \in \mathbb{R}^n$

$$\frac{t}{2}.\sum_{i=1}^{M}\sum_{j=1}^{N}(y_{i,j}-x_{i,j})^2 \;+\; \sum_{i=1}^{M-1}\sum_{j=1}^{N}|y_{i+1,j}-y_{i,j}| \;+\; \sum_{i=1}^{N}\sum_{j=1}^{M-1}|y_{i,j+1}-y_{i,j}| \tag{1}$$

To simplify notations we denote by $\mathcal{P}$ the set of all pixels, i.e,

$$\mathcal{P} = \{(i,j)\,|\,i = 1,\ldots,M; j = 1,\ldots,N\}.$$

For each pixel $(i,j) \in \mathcal{P}$ we denote by $\mathcal{N}_{i,j}^{+}$ the set of "right" and "bottom" neighbors of (i,j) defined by

$$\mathcal{N}_{i,j}^{+} \;=\; \left\{(i',j') \in \mathcal{P} \mid \left(i' = i+1 \text{ and } j' = j\right) \text{ or } \left(i' = i \text{ and } j' = j+1\right)\right\},$$

and by $\mathcal{N}_{i,j}^{-}$ the set of "left" and "top" neighbors of (i,j) defined by

$$\mathcal{N}_{i,j}^{-} = \left\{(i',j') \in \mathcal{P} \mid \left(i' = i-1 \text{ and } j' = j\right) \text{ or } \left(i' = i \text{ and } j' = j-1\right)\right\}.$$

Thus (1) can be rewritten as follows

$$\frac{t}{2}\sum_{p\in\mathcal{P}}(y_p - x_p)^2 + \sum_{p\in\mathcal{P}}\sum_{q\in\mathcal{N}_p^+}|y_q - y_p|.$$

We proceed as we did in class. We use splitting to get an equivalent problem

$$\begin{cases} \min\limits_{y\in\mathbb{R}^{MN}} \min\limits_{d\in\mathbb{R}^{M(N-1)+(M-1)N}} & \dfrac{t}{2}\displaystyle\sum_{p\in\mathcal{P}}(y_p - x_p)^2 + \sum_{p\in\mathcal{P}}\sum_{q\in\mathcal{N}_p^+}|d_{p,q}| \\ \text{subject to} & d_{p,q} - (y_q - y_p) = 0 \quad \text{for any } p \in \mathcal{P}, q \in \mathcal{N}_p^+ \end{cases}$$

Note that we have also abused notation for $d \in \mathbb{R}^{M(N-1)+(M-1)N}$ since we index the entries of this vector by (p,q). We will also abuse notation the Lagrange multipliers $\lambda \in \mathbb{R}^{M(N-1)+(M-1)N}$ and denote by $\lambda_{p,q}$ the Lagrange multiplier associated to the constraint $d_{p,q} - (y_q - y_p) = 0$.

Then we introduce an Augmented Lagrangian $\mathcal{L} : \mathbb{R}^{MN}\times\mathbb{R}^{M(N-1)+(M-1)N}\times\mathbb{R}^{M(N-1)+(M-1)N} \to \mathbb{R}$ defined by

$$\begin{aligned}\mathcal{L}(y,d,\lambda) =& \frac{t}{2}\sum_{p\in\mathcal{P}}(y_p - x_p)^2 + \sum_{p\in\mathcal{P}}\sum_{q\in\mathcal{N}_p^+}|d_{p,q}| \\ &+ \frac{1}{2}\sum_{p\in\mathcal{P}}\sum_{q\in\mathcal{N}_p^+}(\lambda_{p,q} + d_{p,q} - (y_q - y_p))^2 - \frac{1}{2}\sum_{p\in\mathcal{P}}\sum_{q\in\mathcal{N}_p^+}(\lambda_{p,q})^2.\end{aligned}$$

We use ADMM to find an approximate saddle point of $\mathcal{L}((\cdot,\cdot),\cdot)$. So we consider 3 sequences $(y^k)_{k\in\mathbb{N}} \subset \mathbb{R}^{MN}$, $(d^k)_{k\in\mathbb{N}} \subset \mathbb{R}^{M(N-1)+(M-1)N}$, and $(\lambda^k)_{k\in\mathbb{N}} \subset \mathbb{R}^{M(N-1)+(M-1)N}$ defined as follows:

1. Initialization:
   (a) $y^0 = x$
   (b) $d^0 = 0$
   (c) $\lambda^0 = 0$
2. Iterate until a stopping criteria is met
   (a) $d^{k+1} = \arg\min_{d \in \mathbb{R}^{M(N-1)+(M-1)N}} \mathcal{L}(y^k, d, \lambda^k)$
   (b) $y^{k+1} = \arg\min_{y \in \mathbb{R}^{MN}} \mathcal{L}(y, d^{k+1}, \lambda^k)$
   (c) for any $p \in \mathcal{P}$ for any $q \in \mathcal{N}_p^+$

$$\lambda_{p,q}^{k+1} = \lambda_{p_q}^k + d_{p,q}^{k+1} - (y_q^{k+1} - y_p^{k+1})$$

Stopping rule: we will stop iterations as soon as $\|y^{k+1} - y^k\|_2 \leq \frac{\|x\|_2}{1000}$.

Numerical approximation of the minimizer: We choose the last computed y^k as the numerical approximation of the minimizer

A data structure for an implementation. We can use the following data structures to perform computations.

- x is 2D array with M lines and N columns. We access the value of the pixel (i, j) of x by x(i,j) for any $1 \leq i \leq M$, $1 \leq j \leq N$.
- y_k is 2D array with M lines and N columns. We access the value of the pixel (i, j) of y^k by y_k(i,j).
- To represent $d \in \mathbb{R}^{M(N-1)+(M-1)N}$ we can use two 2D arrays:
  - d_right_k with M lines and $(N-1)$ columns. The value d_right_k(i,j) corresponds to $d_{p,q}^k$ with $p = (i, j)$ and $q = (i, j+1)$, for any $1 \leq i \leq M$, $1 \leq j \leq (N-1)$.
  - d_bottom_k with $(M-1)$ lines and N columns. The value d_bottom_k(i,j) corresponds to $d_{p,q}^k$ with $p = (i, j)$ and $q = (i+1, j)$, for any $1 \leq i \leq (M-1)$, $1 \leq j \leq N$.
- To represent $\lambda^k \in \mathbb{R}^{M(N-1)+(M-1)N}$ we can use two 2D arrays:
  - lambda_right_k with M lines and $(N-1)$ columns. The value lambda_right_k(i,j) corresponds to $\lambda_{p,q}^k$ with $p = (i, j)$ and $q = (i, j+1)$, for any $1 \leq i \leq M$, $1 \leq j \leq (N-1)$.
  - lambda_bottom_k with $(M-1)$ lines and N columns. The value lambda_bottom_k(i,j) corresponds to $\lambda_{p,q}^k$ with $p = (i, j)$ and $q = (i+1, j)$, for any $1 \leq i \leq (M-1)$, $1 \leq j \leq N$.

We now give the pseudo-code to implement the update of d^k, y^k and λ^k.

Update for d^k

To perform the update of d^k we need to implement the proximal operator for the absolute value. The pseudo-code is as follows

1: Input: z real value
2: Output: *out*

```
function PROX_ABSOLUTE_VALUE(z)
    out ← 0
    if z ≥ 1 then
        out ← z − 1
    end if
    if z ≤ −1 then
        out ← z + 1
    end if
    return out
end function
```

The pseudo for updating d^k is as follows

```
Inputs: lambda_bottom_k, lambda_right_k, y_k
Outputs: d_bottom_kp1, d_right_kp1
function UPDATE_D(lambda_bottom_k, lambda_right_k, y_k)
    for i = 1 do to M − 1
        for j = 1 do to N
            d_bottom_kp1(i,j) = Prox_absolute_value(y_k(i+1, j) - y_k(i,j) - lambda_bottom_k(i,j))
        end for
    end for
    for i = 1 do to M
        for j = 1 do to N − 1
            d_right_kp1(i,j) = Prox_absolute_value(y_k(i, j+1) - y_k(i,j) - lambda_right_k(i,j))
        end for
    end for
    return (d_bottom_kp1, d_right_kp1)
end function
```

Update for y^k

As mentioned in class, we do not solve exactly the optimization problem for the update of y^k. Instead we approximately solve the optimization problem by running 5 iterations of Gauss-Seidel method.

The psuedo-code is as follows:

```
Inputs: y, lambda_bottom, lambda_right, d_bottom, d_right, x, t
Outputs: y_p1
function UPDATE_Y(y, lambda_bottom, lambda_right, d_bottom, d_right, x, t)
    y_p1 ← y
    for k = 1 do to 5
        for i = 1 do to M
            for j = 1 do to N
                nb_neighbors=0
                %% Cope with boundary values for y
                y_im1_j=0;
                if (i>1) y_im1_j = y_p1(i-1,j ); nb_neighbors++; end if
                y_ip1_j=0;
                if (i<Nrows) y_ip1_j = y_p1(i+1,j); nb_neighbors++; end if
                y_i_jm1=0;
                if (j>1) y_i_jm1 = y_p1(i ,j-1); nb_neighbors++; end if
```

```
16:                 y_i_jp1=0;
17:                 if (j<Ncols) y_i_jp1 = y_p1(i ,j+1);nb_neighbors++; end if
18:
19:                 %% Cope with boundary values for d_right and d_bottow
20:                 dr_i_j=0;
21:                 if (j<Ncols) dr_i_j=d_right(i,j); end if
22:                 dd_i_j=0;
23:                 if (i<Nrows) dd_i_j=d_bottom(i,j); end if
24:                 dr_i_jm1=0;
25:                 if (j>1) dr_i_jm1 = d_right(i,j-1); end if
26:                 dd_im1_j=0;
27:                 if (i>1) dd_im1_j = d_bottom(i-1,j); end if
28:
29:                 %% Cope with boundary values for lambda_right and lambda_bottow
30:                 lr_i_j=0;
31:                 if (j<Ncols) lr_i_j=lambda_right(i,j); end if
32:                 ld_i_j=0;
33:                 if (i<Nrows) ld_i_j=lambda_bottom(i,j); end if
34:                 lr_i_jm1=0;
35:                 if (j>1) lr_i_jm1 = lambda_right(i,j-1); end if
36:                 ld_im1_j=0;
37:                 if (i>1) ld_im1_j = lambda_bottom(i-1,j); end if
38:
39:                 % perform computations
40:                 tmp_y = y_im1_j + y_ip1_j + y_i_jm1 + y_i_jp1
41:                 tmp_d = +dr_i_jm1 - dr_i_j + dd_im1_j - dd_i_j
42:                 tmp_l = +lr_i_jm1 - lr_i_j + ld_im1_j - ld_i_j;
43:                 tmp = t*x(i,j) + tmp_y + (tmp_l + tmp_d);
44:                 tmp = tmp / (t + nb_neighbors);
45:                 y_p1(i,j) = tmp;
46:             end for
47:         end for
48:     end for
49:     return (y_p1)
50: end function
```

Update for λ^k

The pseudo for updating λ^k is as follows

```
1: Inputs: lambda_bottom_k, lambda_right_k, d_bottom_k, d_right_k, y_k
2: Outputs: lambda_bottom_kp1, lambda_right_kp1
3: function UPDATE_LAMBDA(lambda_bottom_k, lambda_right_k, y_k)
4:     for i = 1 do to M − 1
5:         for j = 1 do to N
6:             lambda_bottom_kp1(i,j) = lambda_bottom_k(i,j) + d_bottom_kp1(i,j) - (y_kp1(i+1,j)
   - y_kp1(i,j))
7:         end for
8:     end for
9:     for i = 1 do to M
```

10: **for** $j = 1$ **do** to $N - 1$
11: lambda_right_kp1(i,j) = lambda_right_k(i,j) + d_right_kp1(i,j) - (y_kp1(i,j+1) - y_kp1(i,j))
12: **end for**
13: **end for**
14: return (lambda_bottom_kp1, lambda_right_kp1)
15: **end function**

What you have to do.
All the files you create must be in one folder/directory.

1. Implement "prox_absolute_value" as a function in one file.
2. Implement "update_d" as a function in one file.
3. Implement "update_y" as a function in one file.
4. Implement "update_lambda" as a function in one file.
5. Implement ADDM as a function in one file (with the given initial condition and stopping criteria)
6. We consider the digital image 'ima-noisy.pgm'. (See the remark below to read, write and displaly this digital image). We consider a the following parameters $t_1 = 5$, $t_2 = 10$, $t_3 = 15$, $t_4 = 20$, $t_5 = 25$, $t_6 = 30$, $t_7 = 35$, $t_8 = 40$. For each value of t_i $(i = 1, \ldots, 8)$. We respectively denote by k_i^* and $y^{k_i^*}$ the number of interations and the denoised image when the parameter t_i is used.
7. Write a Matlab script that computes k_i^*, $y^{k_i^*}$ and $\|y^{k_i^*}\|_2$ for the given values of t_i $(i = 1, \ldots, 8)$.
8. Create a PDF that contains gives for each t_i the number of iterations k_i^* and $\|y^{k_i^*}\|_2$.
9. Save the "denoised" image y^{k*_i} for $t_i = 20$ that you have computed as "denoised.pgm".

A remark about reading/writing and displaying an image.

- There are several formats for encoding digital images. The image I provide is encoded using the PNM standard. Each pixel is a integer between 0 and 255. Therefore after reading the image we need to divide each value by 255. In Octave/matlab, you can read the image using the following code
x = double(imread('ima-noisy.pgm'))/255;
Make sure that all pixel values are in [0,1].

- To display the image x you can use the following code
imshow(x)

- The following code write x with all values in [0,1] as an image file named "x-write.pgm" (PNM format).
imwrite(int_u8(x*255), 'x-write.pgm')